



**UNIVERSITATEA DIN BUCUREȘTI**  
**FACULTATEA DE**  
**MATEMATICĂ ȘI INFORMATICĂ**



**SPECIALIZAREA INFORMATICĂ**

**Disertație**

**OPEN SOURCE RUNTIME SECURITY  
TOOLS IN AZURE KUBERNETES  
SERVICE**

**Absolvent**  
**Radu-Constantin Onuțu**

**Coordonator științific**  
**Lect. dr. Silviu Laurențiu Vasile**

**București, iulie 2026**

## **Rezumat**

Această lucrare de disertație prezintă o evaluare comparativă a trei instrumente open source bazate pe eBPF: Falco, Tetragon și Tracee. Evaluarea adoptă framework-ul MITRE ATT&CK for Containers, cu 18 teste aplicate pe Azure Kubernetes Service (AKS), ce acoperă toate cele zece tactici ale framework-ului. Metricile cantitative de detecție (timp mediu de detecție, rată de detecție, profilul alertelor fals pozitive, consum de CPU și memorie) sunt combinate cu metrici calitative privind calitatea alertelor, capacitatea de aplicare a regulilor și complexitatea operațională. Cele trei instrumente obțin rate de detecție comparabile, cu timpi medii de răspuns de aproximativ o secundă, dar diferă semnificativ în ceea ce privește efortul de scriere a regulilor, gradul de detalii al metadatelor alertelor, profilul alertelor fals pozitive și posibilitatea aplicării regulilor la nivel de kernel. Concluziile oferă sugestii pentru selecția instrumentelor în AKS.

## **Abstract**

This master's thesis presents a comparative evaluation of three open source eBPF based tools: Falco, Tetragon, and Tracee. The evaluation adopts the MITRE ATT&CK for Containers framework, with 18 tests run on Azure Kubernetes Service (AKS), covering all ten tactics of the framework. Quantitative detection metrics (mean time to detect, detection rate, false positive profile, CPU and memory overhead) are combined with qualitative metrics covering alert quality, rule enforcement capability, and operational complexity. All three tools achieve comparable detection rates with mean response times around one second, but differ significantly on rule authoring effort, alert metadata detail, false positive profile, and the availability of in-kernel rule enforcement. The conclusions offer guidance for tool selection on AKS.

# Contents

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| <b>List of Tables</b>                                           | <b>5</b>  |
| <b>List of Figures</b>                                          | <b>6</b>  |
| <b>1 Introduction</b>                                           | <b>7</b>  |
| 1.1 Problem Context . . . . .                                   | 7         |
| 1.2 Motivation and Research Gap . . . . .                       | 7         |
| 1.3 Research Questions . . . . .                                | 8         |
| 1.4 Contributions . . . . .                                     | 8         |
| 1.5 Thesis Outline . . . . .                                    | 8         |
| <b>2 Background</b>                                             | <b>9</b>  |
| 2.1 eBPF Fundamentals . . . . .                                 | 9         |
| 2.2 Kubernetes Runtime Security Context . . . . .               | 11        |
| 2.3 Tool Architectures . . . . .                                | 11        |
| 2.3.1 Falco . . . . .                                           | 11        |
| 2.3.2 Tetragon . . . . .                                        | 12        |
| 2.3.3 Tracee . . . . .                                          | 13        |
| 2.4 AKS-Specific Considerations . . . . .                       | 14        |
| <b>3 Related Work</b>                                           | <b>14</b> |
| 3.1 eBPF in Container and Cloud Security . . . . .              | 14        |
| 3.2 Comparative Evaluations of Runtime Security Tools . . . . . | 15        |
| 3.3 Research Gap and Thesis Positioning . . . . .               | 15        |
| <b>4 Methodology</b>                                            | <b>16</b> |
| 4.1 Research Approach . . . . .                                 | 16        |
| 4.2 Test Environment . . . . .                                  | 16        |
| 4.2.1 Cluster Topology and Version . . . . .                    | 16        |
| 4.2.2 Network and Admission Configuration . . . . .             | 17        |
| 4.2.3 Supporting Infrastructure . . . . .                       | 18        |
| 4.3 Attack Catalogue . . . . .                                  | 18        |
| 4.3.1 Construction Method . . . . .                             | 18        |
| 4.3.2 Execution Contexts . . . . .                              | 18        |
| 4.3.3 Adaptations and Custom Atomics . . . . .                  | 19        |
| 4.3.4 Scope Exclusions . . . . .                                | 19        |

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| 4.4      | Tool Deployment . . . . .                         | 20        |
| 4.4.1    | Deployment Pattern . . . . .                      | 20        |
| 4.4.2    | Per Tool Deployment . . . . .                     | 21        |
| 4.4.3    | Configuration Equivalence . . . . .               | 21        |
| 4.5      | Runner and Execution Procedure . . . . .          | 22        |
| 4.5.1    | Runner Design . . . . .                           | 22        |
| 4.5.2    | Per Atomic Execution Procedure . . . . .          | 22        |
| 4.5.3    | Alert Attribution and MTTD . . . . .              | 22        |
| 4.6      | Evaluation Metrics . . . . .                      | 23        |
| 4.7      | Threats to Validity . . . . .                     | 24        |
| <b>5</b> | <b>Results</b>                                    | <b>25</b> |
| 5.1      | Overview of the Experimental Runs . . . . .       | 25        |
| 5.2      | Detection Rate . . . . .                          | 25        |
| 5.3      | Mean Time to Detect . . . . .                     | 26        |
| 5.4      | False Positive Behaviour . . . . .                | 27        |
| 5.5      | Operational Complexity . . . . .                  | 29        |
| 5.6      | Resource Overhead . . . . .                       | 30        |
| 5.7      | Alert Quality and Contextual Enrichment . . . . . | 31        |
| 5.8      | Runtime Enforcement . . . . .                     | 33        |
| 5.9      | Cross-tool Summary . . . . .                      | 34        |
| <b>6</b> | <b>Discussion and Conclusion</b>                  | <b>35</b> |
| 6.1      | Synthesis and Research Questions . . . . .        | 35        |
| 6.2      | Tool-Selection Guidance . . . . .                 | 35        |
| 6.3      | Limitations . . . . .                             | 36        |
| 6.4      | Future Work and Concluding Remarks . . . . .      | 36        |
|          | <b>References</b>                                 | <b>37</b> |
| <b>A</b> | <b>Evaluation Metric Definitions</b>              | <b>39</b> |
| <b>B</b> | <b>MTTD per Atomic per Tool</b>                   | <b>42</b> |
| <b>C</b> | <b>Per-Tool Deployment Detail</b>                 | <b>42</b> |

## List of Tables

|     |                                                                                                   |    |
|-----|---------------------------------------------------------------------------------------------------|----|
| 4.1 | Atomic execution contexts. . . . .                                                                | 19 |
| 4.2 | Tool deployment versions. . . . .                                                                 | 20 |
| 5.1 | Per atomic detection rate for the three canonical runs (detected repetitions out of ten). . . . . | 26 |
| 5.2 | Mean Time to Detect distribution per tool, over detected repetitions. . . . .                     | 26 |
| 5.3 | False positive alerts emitted during the 30 min baseline window. . . . .                          | 27 |
| 5.4 | Top FP-generating rules and signatures per tool during the baseline window. . . . .               | 28 |
| 5.5 | Consolidated comparison of operational complexity across the three tools. . . . .                 | 29 |
| 5.6 | Per-pod CPU and memory consumption per tool, baseline mean and attack peak. . . . .               | 31 |
| 5.7 | Alert quality and contextual enrichment per tool. . . . .                                         | 32 |
| 5.8 | Tetragon enforcement demonstration on T1611.1: aggregate outcome over 10 repetitions. . . . .     | 34 |
| 5.9 | Consolidated cross-tool summary, one row per tool and one column per metric. . . . .              | 34 |
| B.1 | MTTD per atomic per tool (seconds). . . . .                                                       | 42 |

## List of Figures

|     |                                                                    |    |
|-----|--------------------------------------------------------------------|----|
| 2.1 | eBPF architecture for runtime security observation. . . . .        | 10 |
| 2.2 | Runtime pipelines of Falco, Tetragon, and Tracee compared. . . . . | 12 |
| 4.1 | Experimental setup on AKS. . . . .                                 | 17 |

# 1 Introduction

## 1.1 Problem Context

Kubernetes has become the dominant container orchestration platform. According to the 2025 CNCF Annual Survey, 82% of container-using organizations now run Kubernetes in production, up from 66% in 2023 [9]. Organizations operating workloads on Kubernetes clusters face runtime security risks that preventive measures such as image scanning and admission control alone cannot address. Once a container is running, it may be exploited through privilege escalation to gain access to the underlying host node, resource hijacking for purposes such as cryptomining, lateral movement across pods and nodes, and other attack techniques catalogued in the MITRE ATT&CK framework for Containers [10]. These runtime threats motivate the need for continuous monitoring and detection within the cluster at the kernel level.

Tools built on the extended Berkeley Packet Filter (eBPF) have emerged as the leading open source approach to runtime security monitoring. By attaching programs to kernel hooks, these tools can observe events in the kernel, including system calls, process execution, file access, and network activity, without modifying application code or requiring loadable kernel modules. Three tools dominate this category: Falco, Tetragon, and Tracee. Falco was originally created by Sysdig and is now a CNCF Graduated project. Tetragon is developed by Isovalent as part of the Cilium project, which is itself CNCF Graduated. Tracee is maintained by Aqua Security as an independent open source project. Although the three tools share a common technological foundation in eBPF, they differ substantially in architecture, detection philosophy, policy model, and enforcement capability.

## 1.2 Motivation and Research Gap

Organisations evaluating open-source runtime security tools for Kubernetes face a practical selection problem. Commercial alternatives (Aqua CNAPP, Sysdig Secure, similar offerings) provide integrated runtime protection, but their licensing costs can be prohibitive for smaller organisations or constrained budgets, creating a strong incentive to evaluate open-source alternatives. The architectural and functional differences among the available tools are not immediately apparent, however, and existing comparative research remains limited. The most directly relevant prior work, Syairozi and Arizal [14], compared Falco, Tetragon, and Tracee on DigitalOcean Managed Kubernetes using three OWASP-derived scenarios, evaluating MTTD, DR, FPR, CPU, and memory across 20 repetitions per scenario; it did not address runtime enforcement testing, alert output quality and contextual enrichment, a broader MITRE ATT&CK Containers-mapped scenario set, deployment on a hyper-scale managed Kubernetes platform, cross-tool rule/policy comparison, or operational-complexity factors such as policy authoring and deployment friction.

The growing adoption of managed Kubernetes further motivates platform-specific evaluation. The major cloud providers, AWS (EKS), Azure (AKS), and Google Cloud (GKE), collectively dominate the IaaS market; a 2025 Gartner report estimates Amazon 37.7%, Microsoft 23.9%, Google 9.0% [5]. Tool behaviour on these managed platforms has direct operational relevance, since compatibility with managed node configurations, kernel versions, and cluster constraints affects both detection capability and deployment complexity.

### 1.3 Research Questions

This master’s thesis addresses the following three research questions:

- **RQ1:** How do Falco, Tetragon, and Tracee differ in their architectural approach to runtime event collection, detection, and response?
- **RQ2:** How do the three tools compare across detection capability, resource overhead, and operational complexity when deployed on Azure Kubernetes Service (AKS)?
- **RQ3:** Based on architectural and empirical findings, what practical guidance can be offered for tool selection in AKS environments?

### 1.4 Contributions

This master’s thesis makes three contributions. First, a quantitative detection-metric evaluation of Falco, Tetragon, and Tracee (MTTD, DR, FPR, CPU, memory) on AKS, a hyperscale managed Kubernetes platform not covered by any prior comparative study of these tools; the metric framework follows Syairozi and Arizal [14], providing continuity with the most directly relevant prior study while extending to a different platform and a broader scenario set. Second, it unifies dimensions previously addressed only in isolation by combining detection metrics with runtime enforcement testing, alert output quality analysis, and operational complexity assessment in a single evaluation. Third, it uses the MITRE ATT&CK for Containers framework as the basis for scenario selection, spanning multiple tactics and extending the attack surface covered relative to the narrower scenario sets in existing comparative work.

### 1.5 Thesis Outline

The remainder of this thesis is structured as follows. Chapter 2 provides background on eBPF, Kubernetes runtime security, and the architectures of the three tools under study. Chapter 3 reviews related work and positions this thesis relative to prior research. Chapter 4 describes the experimental methodology, including the test environment, attack scenarios, and evaluation framework. Chapter 5

presents the empirical results. Chapter 6 discusses the findings, provides tool-selection guidance, addresses limitations, and concludes the thesis with directions for future work.

## 2 Background

### 2.1 eBPF Fundamentals

The extended Berkeley Packet Filter (eBPF) is an in-kernel execution environment that runs sandboxed, event-driven user-defined programs in kernel space [12]. Introduced in 1992 as the Berkeley Packet Filter (BPF) for network packet filtering in tools such as `tcpdump`, classical BPF was extended in 2014 with a richer instruction set, in-kernel data structures, and more attachment points [6]. Now simply eBPF, the extended form has become a general-purpose mechanism for kernel instrumentation, widely used in networking, observability and tracing, and runtime security [13].

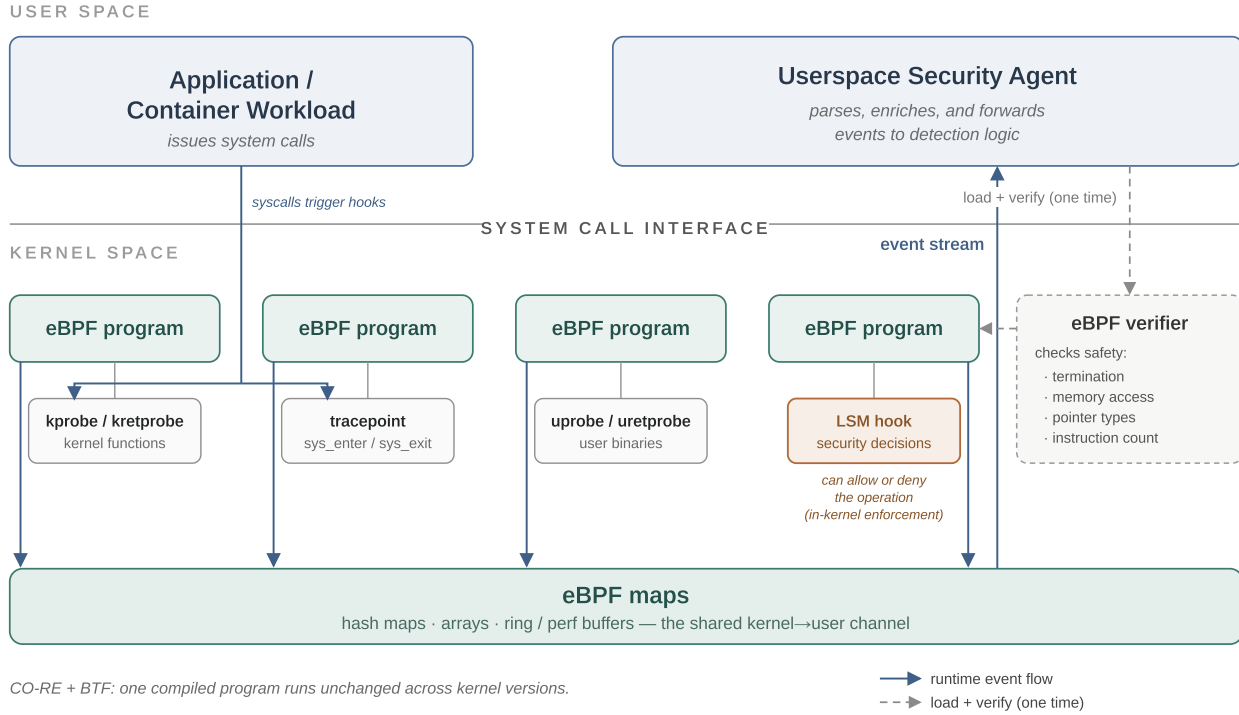
The motivation for eBPF is the difficulty of safely extending kernel functionality: modifying the kernel directly is slow and risky, while loadable kernel modules run with full kernel privileges and can compromise stability. eBPF addresses both by letting programs extend kernel behavior at well-defined hook points, without modifying kernel source or loading modules [12].

eBPF programs are event-driven, executing when the kernel reaches a hook point (Figure 2.1). Several categories matter for runtime security [6]: `kprobes` and `kretprobes` dynamically instrument kernel functions on entry and return; `tracepoints` are static instrumentation points at stable kernel locations, including the `sys_enter` and `sys_exit` tracepoints that observe system call entry and exit; `uprobes` and `uretprobes` are their user-space equivalents, attached to user binaries; and Linux Security Module (LSM) hooks, exposed to eBPF since kernel 5.7 via the BPF LSM subsystem, sit at security-critical decision points and, unlike tracing hooks, can return values that permit or deny the underlying operation [12], the basis for in-kernel enforcement (Subsection 2.3.2). Attachment point also affects portability: `tracepoints` are a stable interface across kernel versions, whereas `kprobes` bind to specific kernel function names that may change between releases, which matters on managed Kubernetes platforms where users do not control node kernels.

Before loading, an eBPF program is checked by the eBPF verifier [12], which statically analyses it to guarantee safe execution in kernel space: it enforces termination, validates memory accesses, checks pointer type safety, and bounds the instruction count, rejecting any program that fails. These guarantees are what allow eBPF code to run safely in kernel context despite originating from userspace.

eBPF programs communicate with userspace and with each other through eBPF maps, kernel-resident key-value structures such as hash maps, arrays, and ring or perf buffers [6]. Runtime security tools use maps as the primary channel for shipping events to a userspace agent that parses,

enriches, and forwards them to detection logic [12]. Every tool examined here follows this two-part architecture (Figure 2.1): eBPF programs attached to kernel hooks plus a userspace agent consuming the event stream.



**Figure 2.1:** eBPF architecture for runtime security observation. The two-part architecture shared by the eBPF-based runtime security tools evaluated in this master’s thesis: eBPF programs attach to kernel hooks (kprobes/kretprobes, tracepoints, uprobes/uretprobes, and LSM hooks) and are admitted only after passing the in-kernel verifier, performed once at load time (dashed path). At runtime (solid arrows) the programs write events to eBPF maps, which a userspace agent consumes, enriches, and forwards to detection logic. Unlike the tracing hooks, the LSM hook can allow or deny the operation in the kernel, the basis for the in-kernel enforcement discussed in Subsection 2.3.2.

A long-standing challenge for eBPF programs has been portability across kernel versions, since eBPF code often accesses internal kernel data structures whose layouts may change between releases. The Compile Once, Run Everywhere (CO-RE) approach addresses this by combining two mechanisms [12]: the BPF Type Format (BTF) embeds kernel type information into the kernel image, providing a machine-readable description of the structures present in the running kernel; and the eBPF compiler emits relocations that the loader resolves at load time against the target kernel’s BTF, adjusting field offsets as needed. The result is that a single compiled eBPF object file can run unchanged across a wide range of kernel versions, which is particularly important in managed Kubernetes environments where node kernel versions are determined by the cloud provider rather than the operator.

## 2.2 Kubernetes Runtime Security Context

Security controls for containerised workloads span three lifecycle phases. *Build-time* controls (image scanning, secret detection, manifest analysis) and *deploy-time* controls (pod security standards, admission controllers, network policies, RBAC) are both preventive, applied before and at scheduling. The *runtime* phase begins once containers execute on cluster nodes: preventive controls have already applied, and the remaining concern is detecting malicious or anomalous behaviour during execution. Runtime security is fundamentally observational, monitoring workloads to generate alerts or, in some cases, enforce responses when suspicious activity is detected.

Runtime observation requires visibility below the Kubernetes API: an attacker executing inside a container interacts with the host kernel through syscalls, process creation, file access, and network operations, which API-level controls cannot observe directly. Traditional host-based agents can, but typically rely on kernel modules or userspace interposition, both carrying performance or stability costs. eBPF-based tools (Section 2.1) instead give continuous low-overhead kernel observation without modifying application code or loading modules, which is what makes them well suited to Kubernetes runtime security.

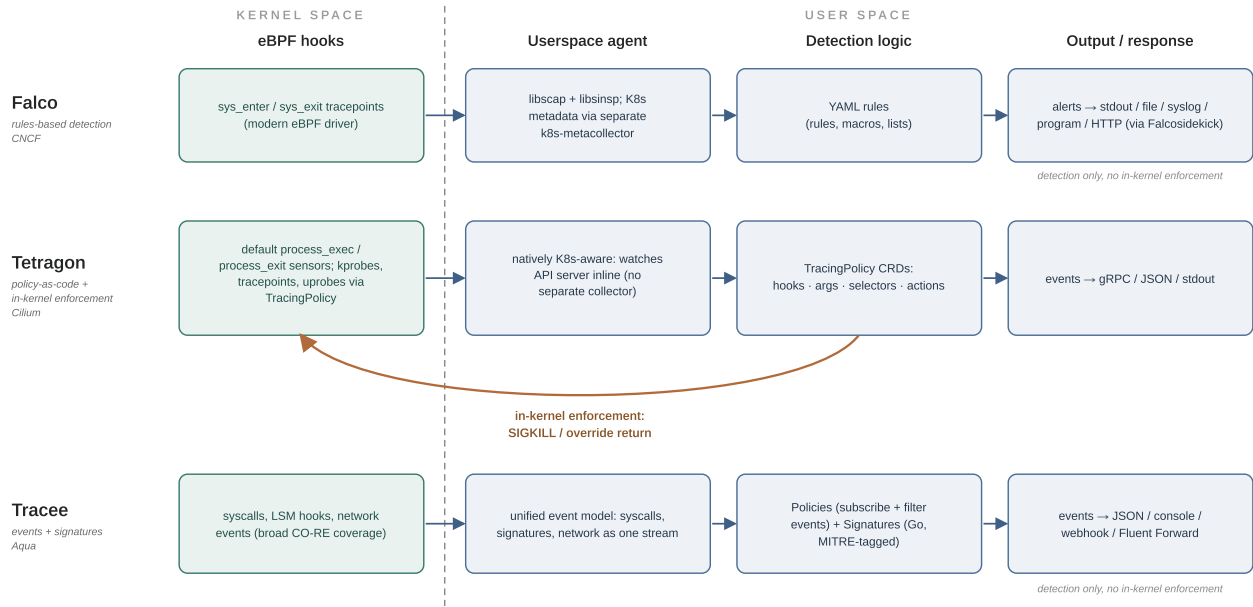
To reason about runtime threats in a structured way, this master’s thesis adopts the MITRE ATT&CK for Containers framework as its threat model [10]. The v19 Containers matrix defines ten tactics (Initial Access, Execution, Persistence, Privilege Escalation, Stealth, Defense Impairment, Credential Access, Discovery, Lateral Movement, Impact), each realised through concrete techniques (e.g. Escape to Host T1611, Resource Hijacking T1496). The evaluation’s attack scenarios (Chapter 4) are drawn from this matrix so the comparison spans all ten tactics rather than a narrow subset; the distribution and coverage rationale are in Section 4.3.

## 2.3 Tool Architectures

All three tools instantiate the two-part eBPF architecture of Section 2.1, kernel hooks feeding a userspace agent, but differ at every stage of the pipeline, most consequentially in whether they can act inside the kernel or only report. Figure 2.2 compares the three pipelines side by side; the subsections below describe each tool in turn, with the specific configuration used for each in this study given in Chapter 4.

### 2.3.1 Falco

Falco is an open-source runtime security tool originally created by Sysdig and donated to the Cloud Native Computing Foundation (CNCF) in 2018. It reached CNCF Graduated status in February 2024, the highest maturity level in the CNCF project hierarchy, indicating broad production adoption and a sustained contributor base [4].



**Figure 2.2:** Runtime pipelines of Falco, Tetragon, and Tracee compared. The runtime pipelines of the three tools, compared across four stages: eBPF hooks, userspace agent, detection logic, and output. All three attach eBPF programs in kernel space and evaluate events in a userspace agent, but only Tetragon can act back in the kernel, terminating the process (SIGKILL) or overriding the syscall return on a TracingPolicy selector match; Falco and Tracee are detection-only.

Falco observes system activity primarily through the `sys_enter` and `sys_exit` tracepoints, captured by one of two driver options (modern eBPF CO-RE or kernel module); this thesis uses the modern eBPF driver, recommended for kernels supporting BTF (Linux 5.8 and later). Its event pipeline is built on `libscap` (driver interface and raw event collection) and `libinsp` (process state tracking, field extraction, and container and Kubernetes enrichment), with Kubernetes metadata supplied by a separate `k8s-metacollector` component that watches the API server so that rules can target specific workloads. Detection logic is expressed in YAML rule files of *rules*, *macros*, and *lists*; a project-maintained default ruleset covers common container and Kubernetes attack patterns. Falco performs no in-kernel enforcement: it can trigger external response actions through program execution or webhook outputs but cannot block or terminate a detected operation at the point of occurrence, the principal architectural contrast with Tetragon (Subsection 2.3.2).

### 2.3.2 Tetragon

Tetragon is an open-source security observability and runtime enforcement tool, developed as a sub-project of Cilium. Cilium is a CNCF Graduated project, and Tetragon was originated by Isovalent [3]. Unlike Falco, which is primarily an alerting tool, Tetragon combines runtime observability with in-kernel enforcement, applying policy and filtering directly inside eBPF programs.

Tetragon’s default sensors emit process lifecycle events (`process_exec` and `process_exit`);

user-defined policies additionally attach to kprobes, tracepoints, uprobes, and, on supporting kernels, LSM hooks (Section 2.1). The agent is natively Kubernetes-aware, watching the API server directly and correlating kernel events with pod, namespace, and workload identity inline rather than through a separate metadata collector as Falco does, and events also carry process ancestry, capabilities, and namespace membership. Detection and enforcement logic is expressed through TracingPolicy CRDs, each specifying the kernel hook points to attach to, the arguments to extract, selectors that filter in-kernel on fields such as process binary path, argument values, pod labels, or namespace (so irrelevant events are discarded before leaving the kernel), and the actions to take on a match. Tetragon is the only tool of the three with native in-kernel enforcement: on a selector match it can terminate the offending process with SIGKILL from inside the eBPF program or override a syscall return via `bpf_override_return` so the call fails without executing, applied directly in the kernel and eliminating the race between detection and response that affects userspace-mediated enforcement.

### 2.3.3 Tracee

Tracee is an open-source runtime security and forensics tool maintained by Aqua Security [1]. Unlike Falco and Tetragon, it is not hosted under the CNCF: it is developed as an independent Aqua Security open-source project. It uses eBPF to observe kernel activity and is positioned primarily as a detection and forensics tool, with built-in mapping of detections to the MITRE ATT&CK framework.

Tracee attaches eBPF programs to a broad set of hooks covering system calls, LSM hooks, and network events, providing out-of-the-box coverage of the full range of Linux system calls plus higher-level security events; like Falco, it uses CO-RE for kernel portability. Its distinguishing architectural property is a unified event model: from version 0.12 onwards, syscalls, signature detections, captured forensic artifacts, and network observations are all emitted as events through a single pipeline, letting downstream consumers and detection logic treat heterogeneous observations uniformly. Tracee separates event subscription from detection: *Policies* (YAML manifests) specify which events a scope subscribes to and filter them on fields such as process name, container, namespace, or argument values, selecting from the existing event catalogue rather than defining new events; detection logic itself lives in *signatures* written in Go, each tagged with the MITRE ATT&CK tactic and technique where applicable. Like Falco, Tracee performs no in-kernel enforcement; its architectural positioning is therefore closer to Falco than to Tetragon, with the additional distinguishing properties of unified event modelling and built-in MITRE ATT&CK mapping.

## 2.4 AKS-Specific Considerations

Azure Kubernetes Service (AKS) is a managed Kubernetes offering in which Microsoft operates the control plane and the customer manages the worker nodes. Because the control plane (API server, scheduler, controller manager, etcd) is not exposed to the customer, user-deployed agents run only on worker nodes; control-plane activity is therefore outside the visibility of Falco, Tetragon, and Tracee on AKS and would have to be observed through Kubernetes audit logging, which is out of scope here (the evaluation is deliberately scoped to node-level eBPF detection, Subsection 4.2.2). AKS offers two Linux node images, Ubuntu and Azure Linux; the image fixes the node kernel version, which governs which eBPF features are available, including CO-RE via BPF Type Format (BTF), BPF LSM, and the enforcement primitives Tetragon uses (Subsection 2.3.2). The specific node image, kernel, and Kubernetes versions are documented in Chapter 4. All three tools ship official Helm charts and run as a DaemonSet, one instance per worker node with access to the node’s kernel event stream, giving consistent configuration across tools for methodological fairness.

## 3 Related Work

### 3.1 eBPF in Container and Cloud Security

eBPF has attracted growing research attention as a foundation for runtime security in containerized environments, with two principal directions: the design of eBPF-based enforcement systems for Kubernetes, and the use of eBPF syscall tracing for detection of specific attack patterns. Gwak, Doan, and Jung [7] propose KRSIE, a Kubernetes runtime instrumentation and enforcement system that compares alternative LSM implementations and uses BPF LSM hooks to control container behaviour at runtime, demonstrating that policy enforcement can be performed directly in the kernel rather than by a userspace intermediary, which is directly relevant here because BPF LSM also underlies Tetragon’s in-kernel enforcement (Subsection 2.3.2). KRSIE itself is a research prototype, not among the tools evaluated here, but its findings support the feasibility of the enforcement architecture that distinguishes Tetragon from Falco and Tracee. Bertinatto et al. [2] take a detection-oriented perspective: they use eBPF to trace syscalls in containerised workloads and apply the resulting event stream to predicting privilege-escalation attacks, confirming that kernel-level visibility via eBPF is sufficient to identify privilege-raising behaviour (one of the MITRE tactics in this thesis’s catalogue). Both studies establish eBPF as a viable foundation for detection and enforcement in Kubernetes runtime security, but focus on individual research systems rather than the mature open-source tools that have since become production defaults. Systematic comparisons of Falco, Tetragon, and Tracee remain scarce.

### 3.2 Comparative Evaluations of Runtime Security Tools

The small body of work that directly compares open-source eBPF-based runtime security tools in Kubernetes consists of three studies, each occupying a distinct position in the evaluation space.

The most directly comparable prior study is Syairozi and Arizal [14], who evaluated Falco, Tetragon, and Tracee on DigitalOcean Managed Kubernetes using three clusters (one per tool) and three OWASP-derived attack scenarios: container escape via `nsenter`, DoS via `stress-ng`, and cryptomining via `xmrig`. Each scenario was repeated 20 times per tool and the authors measured MTTD, DR, FPR, CPU, and memory; all three tools achieved 100% DR and 0% FPR, with Tetragon recording the lowest MTTD for container escape and cryptomining and Falco the lowest for DoS. The study does not address Tetragon’s enforcement capability, alert output quality, a MITRE ATT&CK-structured scenario set, deployment on a major commercial managed Kubernetes offering, or operational complexity factors such as policy authoring effort.

Her et al. [8] conduct a broader qualitative and performance analysis of four tools (KubeArmor, Falco, Tetragon, Tracee), emphasising internal architecture, policy profiling, and micro-benchmarks (process-level via `/usr/bin/ls` execution, network-level via UDP packet processing, plus a policy-enforcement and a log-loss benchmark). The study confirms in-kernel enforcement on Tetragon and KubeArmor and characterises performance trade-offs across the four tools, but does not evaluate detection effectiveness against a controlled scenario set, does not structure its evaluation around MITRE ATT&CK, runs on a self-managed Kubernetes deployment rather than a managed cloud service, and includes KubeArmor (outside the present scope).

Van Vugt and Malik [15] evaluate Falco (both kernel-module and eBPF drivers), Tetragon, KubeArmor, and Tracee against a single simulated Command-and-Control scenario exploiting the API server from a vulnerable deployment. Their methodology emphasises observability quality (log granularity, timestamp presence, identifier completeness) and enforcement capability, alongside resource utilisation and response time, on Grid’5000 and a consumer-grade private server. The scope is bounded by a single attack scenario, and the evaluation platforms do not include a hyperscale managed Kubernetes offering.

### 3.3 Research Gap and Thesis Positioning

Three gaps follow from the prior comparative work. No study evaluates Falco, Tetragon, and Tracee on a hyperscale managed Kubernetes platform (AKS, EKS, GKE), where the provider-determined kernel and node image and the integration surface with cloud-native identity and audit services materially affect deployment and visibility in ways a self-managed or DigitalOcean-based evaluation cannot fully capture. No study organises its scenario set around the MITRE ATT&CK framework for Containers, the standard threat model for containerised environments, rather than the OWASP-

derived or ad-hoc scenario sets used in existing work, which span the tactic space only partially. And no study combines detection effectiveness, runtime enforcement, alert output quality, and operational complexity in a single evaluation scoped to these three tools; each dimension has been addressed in isolation. This master’s thesis addresses the intersection: it applies the detection-metric framework of Syairozi and Arizal [14] to an evaluation on AKS, extends the scenario set to cover a broader range of MITRE ATT&CK tactics, and unifies the evaluation dimensions by combining detection metrics with enforcement testing, alert-quality analysis, and operational-complexity assessment within a single experimental framework.

## **4 Methodology**

### **4.1 Research Approach**

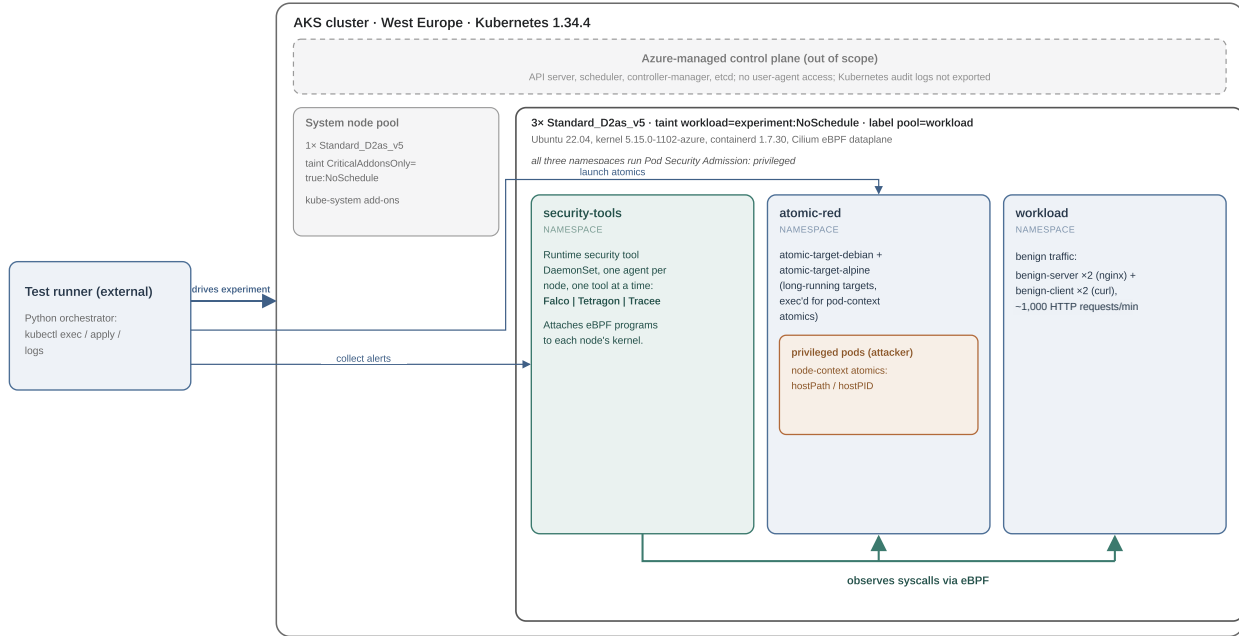
The experimental methodology evaluates Falco, Tetragon, and Tracee on AKS under three commitments. The evaluation is empirical and quantitative for dimensions that admit measurement (detection latency, detection rate, false-positive count, resource utilisation) and qualitative for those that do not (alert quality and contextual enrichment, operational complexity). The apparatus is reproducible end-to-end: every component is captured in version-controlled manifests, every tool version is pinned, and the attack scenarios derive from a public corpus with a recorded commit hash. The comparison runs under deliberately uniform conditions (one tool at a time on the same cluster, kernel, and workload), so that differences in measured outcomes are attributable to the tools.

### **4.2 Test Environment**

#### **4.2.1 Cluster Topology and Version**

The experiments ran on a single Azure Kubernetes Service cluster in West Europe, on Kubernetes 1.34.4 with worker nodes on the AKS Ubuntu 22.04 node image (AKSUbuntu-2204gen2containerd-202603.30.0), Linux kernel 5.15.0-1102-azure, and containerd 1.7.30. All four nodes shared identical kernel and runtime versions, necessary for valid comparison of eBPF based tools across runs.

The cluster used two node pools (Figure 4.1). A single node system pool hosted Kubernetes system components, tainted `CriticalAddonsOnly=true:NoSchedule`. A three node workload pool of identical specification hosted everything experimental (the tool DaemonSet under test, target pods, per-atomic attacker pods, and benign traffic generators), tainted `workload=experiment:NoSchedule` and labelled `pool=workload` so that participating pods had to declare both a matching toleration and node selector.



**Figure 4.1:** Experimental setup on AKS. A runner drives a single AKS cluster (Kubernetes 1.34.4) whose Azure-managed control plane is out of scope. A one node system pool hosts kube-system; a three node workload pool (taint workload=experiment:NoSchedule, label pool=workload) carries the three experimental namespaces: the runtime security tool DaemonSet (security-tools, one agent per node, one tool at a time), the targets and attacker pods (atomic-red), and the continuous benign traffic generators (workload). Each node’s agent observes the syscalls of all pods on that node via eBPF.

## 4.2.2 Network and Admission Configuration

Cluster networking used Azure CNI in overlay mode with Cilium as the network policy engine and eBPF dataplane. Cilium attaches its own eBPF programs on every node, competing for the same kernel attach points as the three tools; the smoke tests in Subsection 4.4.1 confirmed they attached without conflict.

Cluster authentication used Microsoft Entra ID with Azure RBAC. Kubernetes API audit logs were not exported through Azure Diagnostic Settings, which bounds the evaluation to node level eBPF detection paths and excludes mechanisms served by the audit log such as Falco’s k8s\_audit plugin. The boundary applies uniformly to all three tools.

Pod Security Admission was set to privileged on the three experimental namespaces to remove admission control as a confounder: several atomics require privileged pod creation, and rejection at the API layer would stop their syscall-level execution from ever reaching the monitoring tools. Pod-level posture is instead set through securityContext per manifest, so benign workload pods run unprivileged while privileged escape pods declare their requirements explicitly.

### 4.2.3 Supporting Infrastructure

Two long-running target pods in the `atomic-red` namespace support atomics that operate by `kubectl exec` into an existing container (post-compromise threat model): `atomic-target-debian` (`debian:12`) and `atomic-target-alpine` (Alpine 3.20, a contrasting `libc` and minimal toolchain). Sample data (populated `.bash_history`, sample `authorized_keys`) is staged in the Debian target at startup, and the runner pre-seeds any per-atomic state declared in the catalogue (Section 4.3) before each invocation. A benign workload runs continuously in the workload namespace throughout every session: two `nginx` servers and two `curl` clients issuing approximately 1,000 HTTP requests per minute across three endpoints; enough to produce realistic process, file-system, and network-syscall activity without saturating the nodes or obscuring attack signals. Alerts emitted during a no-attack baseline window over this workload are the basis for the false-positive measurement of Section 4.6.

## 4.3 Attack Catalogue

### 4.3.1 Construction Method

The attack scenarios are drawn from Atomic Red Team [11], an open-source library of self-contained scripts that each implement one MITRE technique; every script maps explicitly to a technique identifier and the corpus is version-controlled, so the exact set used is citable by commit hash. Construction applied three filters: (1) retain only techniques in the MITRE ATT&CK for Containers matrix; (2) retain only atomics whose `supported_platforms` includes containers or Linux and whose executor makes sense inside a containerised workload (excluding scripts that assume `systemd`, interactive sessions, a desktop environment, or Active Directory); (3) require each retained atomic to execute cleanly on AKS or admit a documented adaptation preserving the detection signal (Docker-dependent atomics were adapted to Kubernetes pod manifests or replaced with equivalents exercising the same syscall surface). The resulting catalogue contains 18 atomics covering all ten v19 Containers tactics.

### 4.3.2 Execution Contexts

Each atomic is associated with one of two execution contexts that determine how it is launched and which detection surface it exercises (Table 4.1). The pod context covers code execution inside an existing workload: 15 of the 18 atomics are launched via `kubectl exec` into a long-running target pod (`atomic-target-debian` or `atomic-target-alpine`). The node context covers boundary crossings from container to host: the remaining 3 atomics (T1611, T1613, T1685.004) launch privileged pods that interact with host namespaces, the host `containerd` socket, or host paths, and represent the

most consequential class of attack signature, exercising the tools’ ability to detect kernel namespace transitions and cross-namespace process activity.

**Table 4.1:** Atomic execution contexts.

| Context | Threat model                               | Launch mechanism                                                   | Example atomics                 |
|---------|--------------------------------------------|--------------------------------------------------------------------|---------------------------------|
| Pod     | Code execution inside an existing workload | kubectl exec into atomic-target-debian or atomic-target-alpine     | T1078.003, T1098.004, T1552.007 |
| Node    | Escape from a workload pod to the host     | Privileged pod with hostPID, hostPath, or containerd socket access | T1611, T1613, T1685.004         |

### 4.3.3 Adaptations and Custom Atomics

The catalogue applies several categories of adaptation to upstream Atomic Red Team scripts. Three atomics whose upstream variants assume Docker (T1611, T1613, T1046) were rewritten as Kubernetes-native equivalents (privileged pod manifests or in-pod command sequences) that exercise the same kernel-level behaviour, since AKS uses containerd while the underlying syscall surface (privileged pod creation, hostPath mount, namespace traversal, runtime socket access) is unchanged. Several atomics required pre-seeding of state the upstream script assumes already exists (T1098.004 `authorized_keys`, T1070.003 `.bash_history`, T1485 target file, T1543.002 `systemd` unit file, T1685.004 `auditd` installation, T1110.001 bake-in of the `sudo` bruteforce script); the runner stages this state immediately before each invocation and cleans it up afterwards, so the atomic exercises the same detection surface as the upstream variant. T1036.004 requires a one-time compile of its C source inside the target pod. T1550.001 is a custom adaptation with no upstream Linux variant: the pod reads the service account token from `/run/secrets/kubernetes.io/serviceaccount/token` and uses it as a Bearer credential against the API server; the eBPF-observable signal is the in-pod `openat` plus `connect` chain, and the cluster-side audit-log half is deliberately not consumed.

### 4.3.4 Scope Exclusions

Several categories of MITRE technique are deliberately excluded from the catalogue, narrowing the comparison to the node-level eBPF detection surface this thesis evaluates. *Audit-log-only techniques*, where the primary detection signal lives in the Kubernetes API audit log rather than at the

syscall layer (notably T1609 Container Administration Command via `kubectl exec`, T1610 Deploy Container at the API layer, and the upstream `kubectl`-based variants of T1613 and T1552.007), are excluded because AKS Diagnostic Settings are not enabled (Subsection 4.2.2); where the same parent technique admits a node-side variant, that variant is included instead (T1613-2 uses `crictl` against the host containerd socket; T1059.004 covers the in-pod loader phase of T1610). *Network-perimeter techniques* (Exploit Public-Facing Application, External Remote Services, Network and Endpoint Denial of Service) are excluded because their detection surface lies at the WAF, network IDS, or auth-service layer rather than at the kernel hook layer, and *registry- and admission-layer techniques* (Implant Internal Image, Inhibit System Recovery) because theirs lies at the image registry or admission controller. *Real-CVE and human-in-the-loop techniques* (Exploitation for Privilege Escalation, User Execution) are excluded as not meaningfully testable in an automated harness, and *image-build techniques* (Build Image on Host) require `buildah` / `nerdctl` on the node, not present by default on stock AKS Ubuntu nodes. The exclusions apply uniformly to all three tools and do not bias the comparison.

## 4.4 Tool Deployment

### 4.4.1 Deployment Pattern

All three tools were installed via their official Helm charts (the deployment path used in the reference paper and in industrial practice), each as a DaemonSet on the workload pool with one pod per node, tolerating the `workload=experiment:NoSchedule` taint and scheduling only on `pool=workload-`labelled nodes. The tools shared the `security-tools` namespace and were used sequentially: only one tool installed at any given time, with the previous installation removed and a  $\geq 30$  s settling period observed before the next. This prevents two tools' eBPF programs from interfering at overlapping kernel attach points, which would confound the comparison. Each deployment was driven by an idempotent, version-controlled installation script pinning the chart version, values file, and post-install validation. Table 4.2 summarises the chart and image versions used.

**Table 4.2:** Tool deployment versions.

| Tool     | Helm chart version | Application version |
|----------|--------------------|---------------------|
| Falco    | 8.0.2              | 0.43.1              |
| Tetragon | 1.6.1              | 1.6.1               |
| Tracee   | 0.24.1             | 0.24.1              |

#### 4.4.2 Per Tool Deployment

Chart versions are listed in Table 4.2; the per-tool configuration, including the full list of upstream rule packs and the custom rules, TracingPolicies, and Policy CRDs prepared for this study, is given in Appendix C. The summary below records the headline configuration per tool.

**Falco.** Three upstream rule packs loaded together with rule auto-update disabled to fix the rule set against drift for the experiment; seven custom rules authored to cover the five techniques that scored zero against the upstream packs at rule-authoring time (T1046, T1485, T1552.007, T1496, T1069.001), each scoped narrowly to bound baseline noise.

**Tetragon.** Default sensors emit `process_exec/process_exit` but ship no detection policies; nine TracingPolicies adapted from upstream examples covering the signal classes required by the catalogue (file monitoring, symlink resolution, sensitive-binary execution, `setuid/setns/prctl` syscalls, TCP egress, AF\_UNIX connect, sensitive-file deletion); memory limit raised from 1 GiB to 2 GiB to survive kprobe attach with all nine policies loaded.

**Tracee.** The runner reads events directly from `stdout` via `kubectl logs`; the default policies and signature set detect 6/18 catalogue atomics out of the box; nine custom Policy CRDs authored to close the remaining gaps, each subscribing to a built-in event (principally `sched_process_exec`, `security_file_open`, `security_inode_unlink`) with narrow argument filters. No custom Rego or Go signatures were written; the policies activate existing events rather than introduce new detection logic.

#### 4.4.3 Configuration Equivalence

Each tool runs on the configuration that a first time user following the project’s documentation would arrive at, with three augmentations: Falco’s three rule packs were supplemented with seven custom rules covering five techniques the stock packs missed at rule-authoring time, and rule auto update was disabled to fix the rule set against drift for the duration of the experiment; Tetragon required nine custom TracingPolicies to enable detection at all (the chart ships no detection policies by default); Tracee’s default policies and signature set were supplemented with nine custom Policy CRDs to cover catalogue atomics the stock configuration missed. Resource configuration was likewise held equivalent: all three tools ran under an identical resource envelope (requests of 200m CPU and 512 MiB memory, limits of 2 vCPU and 2 GiB memory), removing Helm chart packaging as a confounder in the resource-overhead comparison.

## 4.5 Runner and Execution Procedure

### 4.5.1 Runner Design

The experiment is driven by a Python runner that orchestrates per-tool data-collection sessions. Inputs are the catalogue YAML (Section 4.3) and a configuration file specifying tool, baseline window length, and output directory. Outputs are three artefacts: a CSV summary with one row per (atomic, repetition), a JSON Lines file containing normalised alert records from the attack phase, and a CSV of alerts from the baseline phase. The runner does not install or uninstall the tools (that responsibility lies with the per-tool scripts of Section 4.4), so it can be re-run against an existing installation without disturbing it. Each session proceeds through four phases: a 60 s settling phase for eBPF programs to attach and the kubelet to converge; a 30 min baseline phase capturing alerts during continuous benign workload activity (treated as false positives by construction); an attack phase iterating over the catalogue and executing each atomic for its configured repetition count; and a teardown phase that writes artefacts and exits.

### 4.5.2 Per Atomic Execution Procedure

For each atomic invocation, the runner follows a deterministic six-step procedure designed for unambiguous attribution: pre-seed required state, record start timestamp, execute the atomic, wait for the detection window to elapse, collect alerts, and run cleanup plus a cooldown. Pre-seeding applies to atomics that exercise file-write paths but assume the file already exists; the runner stages this state in the target pod via `kubectl exec` immediately before invocation (Subsection 4.3.3). The start timestamp is captured in UTC with millisecond precision on the runner host. The atomic is launched through one of two context-specific paths (pod or node; Subsection 4.3.2). The detection window, configured per atomic in the catalogue from 15 s for low-latency syscall events up to 90 s for the delayed at-job atomic, bounds the interval during which an alert is considered to belong to this atomic. After the window elapses, the runner fetches alerts from the tool’s `DaemonSet` via `kubectl logs --since-time` with 5 s of left-padding to absorb runner/worker clock skew. Cleanup removes atomic-created artefacts (files, transient pods, scheduled jobs) before a 15–30 s cooldown prevents state leaking into the next window.

### 4.5.3 Alert Attribution and MTTD

Each alert is converted to a normalised record by a tool-specific parser that extracts timestamp, rule/policy name, severity, process name and arguments, file path (if any), and pod/namespace from the tool’s JSON output; the JSON shapes differ substantially across tools, but the normalised record exposes a uniform interface to the runner. An alert is attributed to an atomic when its timestamp

falls within the atomic’s detection window *and* it matches at least one of the atomic’s expected signatures (small per-atomic lists of process names, file-path keywords, network keywords, and alert keywords, with conservative OR logic; any matching field counts). To prevent cross-attribution between atomics touching similar resources, the matcher additionally restricts to alerts originating in the `atomic-red` namespace, excluding benign-workload and runner-internal alerts. This is best-effort rather than ground truth; residual risks of false and under-attribution are discussed in Section 4.7.

## 4.6 Evaluation Metrics

The evaluation reports seven metrics. Full construct definitions, including the unit conventions, departures from the reference paper, and the rationale for the false-positive formulation, are given in Appendix A; the summaries below state what each metric measures and how it is reported.

**Mean Time to Detect (MTTD)** is the average elapsed time between atomic start and first matching alert, in seconds, over detected repetitions; misses are excluded from the average and reported separately. MTTD is reported per atomic per tool and aggregated per tool across the catalogue.

**Detection Rate (DR)** is the proportion of repetitions producing at least one matching alert,  $DR = TP / (TP + FN)$ , reported per atomic per tool and aggregated per tool.

**False Positive Behaviour** is reported as the absolute alert count and per-minute rate during a thirty-minute baseline window of continuous benign traffic with no attack atomics (Subsection 4.2.3). The classical FPR formula is not directly applicable to continuous-time monitoring because the true-negative count is not observable; the construct rationale is given in Appendix A.

**Resource Overhead** is per-pod CPU (millicores) and memory (MiB) sampled at one-minute intervals via the Kubernetes Metrics Server, reported as the baseline-window mean and the attack-phase peak. All three tools share identical resource limits (Subsection 4.4.3), so the figures reflect what each tool used within an equal envelope.

**Alert Quality and Contextual Enrichment** is a qualitative assessment of alerts on a sample (one per detecting atomic). Each of seven contextual fields (rule or signature identifier, severity, process name and command line, file path or network endpoint, Kubernetes pod name and namespace, container identifier, and MITRE ATT&CK technique tag) is scored present, partial, or absent; a composite per-tool score accompanies the breakdown.

**Operational Complexity** is assessed along three aspects: deployment friction (operational obstacles encountered during install), policy authoring effort (the configuration artefacts authored for this study to reach the catalogue’s observed coverage), and observability defaults (whether each tool, out of the box, emits alerts on attack patterns at all). These three aspects feed into the tool-selection guidance in Section 6.2.

**Runtime Enforcement** is reported as a qualitative demonstration on Tetragon, the only tool of the three with native in-kernel enforcement (via `matchActions` on `TracingPolicy` selectors); Falco’s core engine and Tracee in the evaluated version are detection-only. Atomic T1611.1 is repeated ten times with the existing `setns` `TracingPolicy` active, and success is termination of the calling process before the targeted syscall completes, attributable to the matching policy. The asymmetry itself is treated as a substantive cross-tool difference surfaced by the comparison.

## 4.7 Threats to Validity

The principal threats to validity are organised along the four standard dimensions; each is mitigated inline at the relevant design point.

**Internal validity.** Default configurations differ across the three projects (Falco ships several hundred rules across three packs, Tetragon ships no detection policies, Tracee ships default policies plus a behavioural signature set); the configuration-equivalence approach of Subsection 4.4.3, running each tool on the configuration a first-time user following its documentation would reach plus the augmentations needed for catalogue coverage, mitigates this. Tetragon’s nine custom `TracingPolicies` nonetheless cover the catalogue’s signal classes by construction, so a lower detection rate on a given atomic reflects the policy set authored for this thesis rather than Tetragon’s underlying capability; this residual risk is bounded by the per-signal-class scoping in Subsection 4.4.2. Alert attribution (Subsection 4.5.3) combines temporal, signature, and namespace scoping; it is best-effort rather than ground truth, with small residual risks of false and under-attribution. Resource limits are equalised at 2 vCPU and 2 GiB across all three tools (Appendix A), removing Helm packaging as a confounder but meaning the overhead figures are not production-grade estimates for chart defaults.

**External validity.** The experiments use a single AKS cluster (Kubernetes 1.34.4, kernel 5.15.0-1102-azure, containerd 1.7.30); behaviour on alternative managed offerings (EKS, GKE), different kernels, or alternative runtimes (CRI-O, gVisor) is not directly tested, so generalisation beyond AKS is hypothesis rather than result. The catalogue covers all ten MITRE ATT&CK for Containers tactics across eighteen Atomic Red Team scripts, broader than any prior comparative study of these tools but still a sample of the technique space; findings are framed in terms of the catalogue rather than the abstract technique space.

**Construct validity.** The false-positive measurement (Section 4.6) departs from the textbook FPR formula because the true-negative count is not observable in continuous-time monitoring; the alternative (absolute count and per-minute rate over a 30 min baseline) is operationally meaningful but not the same construct, and cross-study comparison with the reference paper’s reported FPR is qualified accordingly. The MTTD measurement records  $t_{\text{start}}$  on the runner host and  $t_{\text{alert}}$  from the agent timestamp on the worker node; under NTP synchronisation the residual skew is below the

smallest detection window, but the reported MTTD includes a small uniform measurement overhead not identical to the true in-agent syscall-to-alert latency.

**Reproducibility.** Helm chart versions, Kubernetes and kernel versions, the Atomic Red Team commit hash, and the catalogue YAML are pinned and version-controlled. The principal residual risk is upstream evolution: Tetragon TracingPolicy examples, the Tracee signature pack, and the Falco default ruleset all evolve over time, and a replication with newer rule or policy content may produce different results.

## 5 Results

### 5.1 Overview of the Experimental Runs

Results are drawn from three canonical runs of the runner (Section 4.5), one per tool, conducted in sequence on the AKS cluster of Subsection 4.2.1. Each run executed the 18-atomic catalogue ten times per atomic (180 repetitions per tool, 540 total), preceded by a 30 min baseline window over the benign workload (Subsection 4.2.3). The figures in this section are computed from the runner’s per-run outputs: a per-repetition summary (detection outcome, MTTD, alert count, matched rules), the normalised alerts captured during the attack phase, and the alerts captured during the baseline window.

### 5.2 Detection Rate

Aggregate detection rate, computed as defined in Section 4.6 from the per-repetition detection outcomes, is 177/180 for Falco (98.3%), 164/180 for Tetragon (91.1%), and 179/180 for Tracee (99.4%). All 540 repetitions completed with no runner errors. Table 5.1 presents the per atomic breakdown. Atomics are grouped by their MITRE ATT&CK for Containers tactic in the order used by the catalogue (Section 4.3); each cell reports the number of detected repetitions out of ten.

The shortfalls below 10/10 are distributed unevenly across the three tools. Falco missed a single repetition on each of three atomics (T1059.004, T1069.001, T1611), and the remaining fifteen atomics detected at every repetition. Tracee missed a single repetition on T1098.004, with the other seventeen atomics in the canonical run detecting at every repetition. Tetragon’s shortfalls span ten atomics: the per atomic floor is 7/10 on T1110.001 (the sudo brute force atomic, with three repetitions missed), four atomics detect at 8/10 (T1053.002, T1046, T1613, T1550.001), and five atomics detect at 9/10 (T1059.004, T1543.002, T1036.004, T1685.004, T1485).

**Table 5.1:** Per atomic detection rate for the three canonical runs (detected repetitions out of ten).

| <b>Tactic</b>        | <b>Atomic</b> | <b>Falco</b>   | <b>Tetragon</b> | <b>Tracee</b>  |
|----------------------|---------------|----------------|-----------------|----------------|
| Initial Access       | T1078.003     | 10/10          | 10/10           | 10/10          |
| Execution            | T1059.004     | 9/10           | 9/10            | 10/10          |
| Execution            | T1053.002     | 10/10          | 8/10            | 10/10          |
| Persistence          | T1098.004     | 10/10          | 10/10           | 9/10           |
| Persistence          | T1136.001     | 10/10          | 10/10           | 10/10          |
| Persistence          | T1543.002     | 10/10          | 9/10            | 10/10          |
| Privilege Escalation | T1611         | 9/10           | 10/10           | 10/10          |
| Stealth              | T1070.003     | 10/10          | 10/10           | 10/10          |
| Stealth              | T1036.004     | 10/10          | 9/10            | 10/10          |
| Defense Impairment   | T1685.004     | 10/10          | 9/10            | 10/10          |
| Credential Access    | T1110.001     | 10/10          | 7/10            | 10/10          |
| Credential Access    | T1552.007     | 10/10          | 10/10           | 10/10          |
| Discovery            | T1046         | 10/10          | 8/10            | 10/10          |
| Discovery            | T1069.001     | 9/10           | 10/10           | 10/10          |
| Discovery            | T1613         | 10/10          | 8/10            | 10/10          |
| Lateral Movement     | T1550.001     | 10/10          | 8/10            | 10/10          |
| Impact               | T1485         | 10/10          | 9/10            | 10/10          |
| Impact               | T1496         | 10/10          | 10/10           | 10/10          |
| <b>Total</b>         |               | <b>177/180</b> | <b>164/180</b>  | <b>179/180</b> |

### 5.3 Mean Time to Detect

MTTD is computed per repetition as defined in Section 4.6, with missed repetitions excluded. Table 5.2 reports the per-tool distribution (mean, median, standard deviation, p95, p99, max) across all detected repetitions in the canonical run; the per-atomic mean per tool is in Appendix B. The unweighted mean across detected repetitions is 1.10 s for Falco, 1.22 s for Tetragon, 1.28 s for Tracee.

**Table 5.2:** Mean Time to Detect distribution per tool, over detected repetitions.

| Tool     | <i>n</i> | Mean (s) | Median (s) | Std. dev. (s) | p95 (s) | p99 (s) | Max (s) |
|----------|----------|----------|------------|---------------|---------|---------|---------|
| Falco    | 177      | 1.10     | 0.95       | 0.39          | 2.01    | 2.44    | 2.48    |
| Tetragon | 164      | 1.22     | 1.07       | 0.54          | 2.25    | 2.69    | 3.51    |
| Tracee   | 179      | 1.28     | 0.87       | 1.96          | 4.04    | 9.70    | 14.97   |

Three patterns are evident in the per-atomic figures (Table B.1, Appendix B). First, Tracee’s MTTD distribution is bimodal: atomics caught on `sched_process_exec` or `security_file_open` hooks (e.g. T1485, T1136.001, T1059.004, T1550.001) report means in the tens of milliseconds, consistent with the policy firing on the kernel event the atomic produces,

while atomics whose detection depends on runner-window aggregation rather than a single hook (T1046) report means above one second. Second, Falco and Tetragon’s MTTD distributions are dominated by the syscall plus userspace rule-evaluation path and cluster more tightly around one second, with little dependence on the atomic; the exceptions are the node-context atomics (T1611, T1613, T1685.004), whose means rise above 1.5 s for all three tools because the runner’s clock starts before the privileged pod is scheduled and started, the same pod-startup latency seen in Section 5.8, not slower agent processing. Third, within-cell variability is small in absolute terms: standard deviations sit below 200 ms in roughly three quarters of the cells and below half a second everywhere except three Tracee cells (T1543.002 at 4.4 s, T1685.004 at 2.5 s, T1110.001 at 1.0 s), whose detection rides on delayed or indirect signals rather than on the immediate kernel event the atomic produces. The clock-skew construct deviation discussed in Section 4.7 is below within-cell variability and is not addressed further here.

## 5.4 False Positive Behaviour

The thirty minute baseline window emitted 31,625 alerts for Falco, 1,533 alerts for Tetragon, and 6,458 alerts for Tracee, corresponding to per minute rates of 1,054, 51.1, and 215.3 respectively. The aggregate counts are dominated by workload generator artefacts in two of the three tools, so the headline figure is uninformative on its own and a per namespace, per signature breakdown is required.

**Table 5.3:** False positive alerts emitted during the 30 min (1800 s) baseline window. *Workload-gen* are alerts attributable to the continuous benign HTTP traffic from the *benign-client* pods in the *workload* namespace; *Environmental* are alerts from system components, host processes, or cloud control-plane endpoints; *atomic-red* reports alerts mis-attributed to the *attack* namespace during baseline.

| Tool     | Total  | Workload-gen   | Environmental | atomic-red |
|----------|--------|----------------|---------------|------------|
| Falco    | 31,625 | 31,515 (99.7%) | 110 (0.3%)    | 0          |
| Tetragon | 1,533  | 1,509 (98.4%)  | 24 (1.6%)     | 0          |
| Tracee   | 6,458  | 3,615 (56.0%)  | 2,843 (44.0%) | 0          |

Tables 5.3 and 5.4 disaggregate the baseline alerts by namespace and signature. Falco’s dominant source is the “Drop and execute new binary in container” rule firing on the *workload* namespace: the benign HTTP client pod (Alpine 3.20) runs `apk add curl` at container start, producing a binary-creation event that the rule then matches on every subsequent benign HTTP request, accounting for 31,515 of 31,625 baseline alerts. The residual 110 alerts split between *kube-system* (18, attributable to *cloud-node-manager* contacting the Azure Instance Metadata Service) and host-level (92, Azure DNS lookups); both are genuine background FPs from pre-existing AKS components, not the experiment. Tetragon’s dominant source is the *tcp-connect-egress* TracingPolicy

**Table 5.4:** Top FP-generating rules and signatures per tool during the baseline window. Counts are alert emissions over 1800 s.

| Tool     | Rule / Signature                                     | Count  |
|----------|------------------------------------------------------|--------|
| Falco    | Drop and execute new binary in container             | 31,515 |
|          | Launch Suspicious Network Tool on Host               | 72     |
|          | Unexpected UDP Traffic                               | 20     |
|          | Contact EC2 Instance Metadata Service From Container | 18     |
| Tetragon | tcp-connect-egress                                   | 1,509  |
|          | file-monitoring                                      | 16     |
|          | follow-symlink                                       | 8      |
| Tracee   | sched_process_exec@exec-http-clients                 | 6,148  |
|          | security_file_open@audit-sa-token                    | 158    |
|          | security_file_open@audit-ssh-private-keys            | 146    |
|          | security_file_open@audit-shadow                      | 4      |
|          | sched_process_exec@exec-uname-pod                    | 1      |
|          | TRC-109                                              | 1      |

firing on every benign HTTP request (1,509 of 1,533); the remaining 24 alerts originate in the `security-tools` namespace, from Tetragon’s own components performing periodic health checks and metadata reads that the `file-monitoring` and `follow-symlink` policies match incidentally.

Tracee’s dominant source is the `exec-http-clients` policy, which fires on every `curl` process start and accounts for 6,148 of the 6,458 baseline alerts; unlike Falco and Tetragon, this source is not purely a workload-generator artefact, since only 3,615 of those executions are the benign client’s requests to `benign-server` while the remaining 2,533 are the AKS node’s own `curl` calls, predominantly to the Azure Instance Metadata Service and the `kubelet healthz` endpoint. After removing the workload-generator share, the 2,843-alert environmental residual is dominated by those host-level `curl` executions (2,533), with the balance from `service-account-token` (158), `SSH-private-key` (146), and `/etc/shadow` (4) read signatures and two single-alert detections.

None of the three tools emitted any baseline alert in the `atomic-red` namespace, where the experiment’s attack pods reside; combined with the contained residual after stripping workload-generator noise, this indicates that no false detection is attributable to the attack target’s own activity for any of the three tools. As noted in Section 4.6, this measurement deviates from the textbook FPR formulation because the true-negative count is not observable in continuous-time monitoring; the figures should be read as alert volume and rate against a fixed benign workload over a fixed window, not as a classification-theoretic FPR, and cross-study comparison with the reference paper’s reported 0% FPR is qualified accordingly.

## 5.5 Operational Complexity

Operational complexity is reported as defined in Section 4.6 across three dimensions: deployment friction (obstacles encountered during install), policy authoring effort (configuration artefacts authored for catalogue coverage), and observability defaults (what is detected at zero configuration). Table 5.5 summarises the dimensions per tool; the three paragraphs below add the analytical commentary the table cannot carry.

On deployment friction, Falco was the cleanest install: the modern eBPF driver attached on the first attempt against AKS Ubuntu 22.04 / kernel 5.15 with no compilation step, kernel-header dependency, or driver build. Tetragon required raising the memory limit from 1 GiB to 2 GiB to survive kprobe attach once the nine TracingPolicies were loaded, and renaming upstream policy examples that share `metadata.name` (several upstream examples use `name: syscalls`) before applying them together. Tracee required applying each Policy CRD via a separate `kubectl apply` after Helm completes, since the chart provides no hook for custom policies (undocumented in the chart values reference), and working around the chart’s flapping readiness probe (pods report Running but never Ready, defeating `helm --wait`).

**Table 5.5:** Consolidated comparison of operational complexity across the three tools, summarising the three dimensions. The full discussion is in the body of the section.

| Dimension                                     | Falco                                      | Tetragon                                                                                                                                                                 | Tracee                                                                                                                                        |
|-----------------------------------------------|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Deployment friction                           | None.                                      | Memory limit raised 1 GiB → 2 GiB to survive kprobe attach; upstream policy examples share <code>metadata.name</code> and must be renamed before being applied together. | Policy CRDs require a separate <code>kubectl apply</code> after Helm; the chart’s readiness probe flaps, defeating <code>helm --wait</code> . |
| Policy authoring effort                       | 7 custom rules in the Falco rule language. | 9 TracingPolicies adapted from upstream examples (kprobe attachments).                                                                                                   | 9 custom Policy CRDs subscribing to built-in events with argument filters.                                                                    |
| Catalogue coverage at install (of 18 atomics) | 14/18 from three upstream rule packs.      | 0/18: default sensors emit lifecycle telemetry, no detections.                                                                                                           | 6/18 from the default policies and signature set.                                                                                             |

On policy authoring effort, the three counts (7 Falco rules, 9 Tetragon TracingPolicies, 9 Tracee Policy CRDs) are similar in scale but differ markedly in nature. Falco rules are written in the

Falco rule language and follow the same shape as the upstream rule packs they supplement, so the mental model matches the one needed to read the defaults. Tetragon TracingPolicies are kprobe attachments expressed in the CRD schema, requiring an appropriate kernel function to hook (e.g. `security_bprm_creds_for_exec` for binary execution, `security_socket_connect` for socket activity), declared argument types so the verifier can resolve them, and argument matchers using operators such as `Postfix` and `Equal`; the mental model is closer to writing a small eBPF program than a detection rule, and per-policy effort is proportionately higher even when a policy covers a single atomic. Tracee Policy CRDs introduce no new detection logic: each subscribes to a built-in event (Subsection 4.4.2) and applies argument filters; per-policy effort is the lowest of the three, but bounded by Tracee’s event catalogue: a signal not exposed by a built-in event would require writing a Rego or Go signature, which Policy CRDs cannot accommodate.

On observability defaults, Falco fires on common container and Kubernetes attack patterns immediately on install, with the three upstream rule packs covering 14/18 catalogue atomics and the custom rules closing the remaining four gaps. Tracee detects 6/18 at install time from its default policies and signature set, providing partial coverage. Tetragon ships with no detection policies at all: the default sensors emit `process_exec/process_exit` on every process lifecycle event, a firehose of execution telemetry rather than a set of detections, and no alert is generated on attack patterns until a TracingPolicy is applied. The asymmetry has practical consequences: a first-time Falco user obtains detection capability immediately, a first-time Tetragon user obtains telemetry but must author policies before producing actionable output, and Tracee sits between with partial coverage at install and a low marginal cost to close the gaps.

## 5.6 Resource Overhead

Resource overhead is reported as defined in Section 4.6. The numbers presented here come from a follow up sampling pass rather than from the canonical detection runs of Section 5.1: the runner does not embed a metrics sampler, and adding one would have required rerunning the full catalogue against each tool. The follow up pass uses the same tool versions, the same cluster, the same workload pod, and identical resource limits (Subsection 4.4.3) as the canonical runs, but the attack phase is shortened from the full catalogue (roughly four hours) to a fifteen minute burst across six atomics chosen to exercise each tool’s dominant hooks. Sampling cadence, unit conventions, and the per pod summary shape carry over unchanged from the metric definition.

The three sampling passes preserved the one tool at a time invariant from the canonical runs (Subsection 4.4.1) by uninstalling each tool before installing the next. Each tool’s DaemonSet ran with three replicas (one per worker node), and the figures reported below aggregate across the three replicas.

**Table 5.6:** Per-pod CPU and memory consumption per tool, during the baseline window and the attack burst. Baseline columns are the arithmetic mean over baseline samples (with the first per-pod sample dropped as install warm-up) aggregated across the three DaemonSet pods; attack peak columns are the global maximum sample observed during the attack burst, across the three pods. CPU is in millicores (m); memory is in mebibytes (MiB).

| Tool     | Baseline (mean) |           | Attack (peak) |           |
|----------|-----------------|-----------|---------------|-----------|
|          | CPU (m)         | Mem (MiB) | CPU (m)       | Mem (MiB) |
| Falco    | 22.8            | 93.1      | 34            | 98        |
| Tetragon | 18.5            | 201.6     | 26            | 230       |
| Tracee   | 44.9            | 125.5     | 66            | 127       |

Table 5.6 reports per-pod CPU and memory consumption during the two phases. The first per-pod baseline sample is dropped as install warm-up: across all three tools it sits an order of magnitude above its successors (the agent has just finished attaching policies and is performing initial enumeration), and including it would inflate the reported steady state.

The three tools show distinct resource profiles. Tetragon has the lowest baseline CPU (18.5 m per pod) but the highest baseline memory (201.6 MiB per pod), consistent with its TracingPolicies attaching kprobes and allocating eBPF maps eagerly at install time and then doing very little steady-state work until events arrive. Falco has the lowest baseline memory (93.1 MiB) and sits in the middle on baseline CPU (22.8 m). Tracee has the highest baseline CPU (44.9 m), reflecting its always-on event collection across the configured Policy CRDs even when no matching event fires. The attack burst raises CPU on all three tools by single to low-double-digit millicores in absolute terms (Falco +11, Tetragon +7.5, Tracee +21) and roughly 40–50% in proportional terms. Memory is essentially flat under load on Falco (+5 MiB) and Tracee (+1.5 MiB); Tetragon shows a modest rise of roughly 28 MiB per pod (around 14% of baseline), consistent with additional kprobe state being populated as the attack burst exercises hooks the idle window did not. The overall pattern is that kernel-side hooks and eBPF maps are loaded eagerly at install time rather than incrementally, so attack-induced load drives per-event userspace work rather than substantial additional kernel-side allocation.

## 5.7 Alert Quality and Contextual Enrichment

Alert quality is reported across the seven contextual fields defined in Section 4.6. For each tool, a sample of alerts (one per detecting atomic) was drawn from the normalised attack-phase alerts of the canonical runs (Section 5.1) and inspected for the presence of each field. Table 5.7 reports the per-tool breakdown, with a composite score in the bottom row; the breakdown carries more interpretive weight than the composite.

**Table 5.7:** Alert quality and contextual enrichment per tool, scored across the seven fields. Each cell is *Present* (the field appears in every alert in the sample), *Partial* (the field appears in some alerts but not others), or *Absent* (the field is not emitted by the tool). The composite is scored 2 for Present, 1 for Partial, 0 for Absent, out of a maximum of 14.

| Field                              | Falco     | Tetragon  | Tracee    |
|------------------------------------|-----------|-----------|-----------|
| Rule or signature identifier       | Present   | Present   | Present   |
| Severity                           | Present   | Absent    | Partial   |
| Process name and command line      | Present   | Present   | Present   |
| File path or network endpoint      | Present   | Present   | Present   |
| Kubernetes pod name and namespace  | Present   | Present   | Present   |
| Container identifier               | Present   | Present   | Present   |
| MITRE ATT&CK technique tag         | Present   | Absent    | Partial   |
| <b>Composite score (out of 14)</b> | <b>14</b> | <b>10</b> | <b>12</b> |

All three tools emit five fields consistently: rule or signature identifier (which links the alert to the detection logic that authored it), process name and command line plus file path or network endpoint (the core forensic content), and pod metadata plus container identifier (workload attribution that makes the alert actionable). The shared coverage reflects all three tools attaching to the same kernel event sources and consuming the same container-runtime metadata, with Kubernetes enrichment integrated by default.

The two fields on which the tools differ are severity and the MITRE ATT&CK technique tag. Falco emits both on every alert: rules ship with a priority on a fixed eight-level scale and a tag list that typically includes MITRE technique identifiers (e.g. the “Container Service Account Token Read By Shell Tool” rule firing on T1552.007 emits T1552.007, container, filesystem, mitre\_credential\_access alongside priority Warning). Tetragon emits neither on its process\_kprobe events: the TracingPolicy CRD schema has no severity element and the event payload no tag field; both can be inferred downstream from policy\_name (this study’s policies encode technique scope in their names) but neither is present as Tetragon emits it. Tracee emits both, but only on signature findings (TRC-NNNN events), which carry a Severity integer and a MITRE tag in their metadata.Properties block; the custom Policy CRDs authored for this study subscribe to built-in events (sched\_process\_exec, security\_file\_open, security\_inode\_unlink) rather than triggering signature findings, and those plain events carry neither, hence the partial scores.

The composite (14/14 Falco, 10/14 Tetragon, 12/14 Tracee) reflects each tool’s design intent. Falco is foremost a detection engine and ships an alert format optimised for an operator triaging the alert: rule provenance, severity ranking, and threat-framework attribution all included by default. Tetragon is foremost an observability and enforcement platform and ships an event format

optimised for downstream processing by alert routers or SIEM systems, leaving severity classification and threat-framework attribution to those layers. Tracee sits between: its signature subsystem produces richly tagged alerts, its policy subsystem produces telemetry without that metadata. A reader choosing on operational grounds should consult the breakdown rather than the composite, since a downstream pipeline performing its own severity classification and MITRE tagging would render the differences moot.

## 5.8 Runtime Enforcement

Runtime enforcement is reported as defined in Section 4.6: only Tetragon supports in-kernel enforcement (the `matchActions` field of `TracingPolicy` selectors); Falco’s core engine emits alerts but does not block, and Tracee in the evaluated version is detection only, so the result is a qualitative demonstration on Tetragon rather than a comparative metric. The asymmetry, that two of three tools cannot block at the kernel level, is revisited in Section 6.2. The enforcement variant of the detection-only `sys-setns` `TracingPolicy` attaches the same `kprobe` on `sys_setns` as the detection policy used elsewhere in this study, but adds a single `matchActions`: `[{action: Sigkill}]` selector scoped to a calling binary of `/usr/bin/nsenter`. The binary scope is necessary rather than cosmetic: `setns(2)` is invoked on every container start by `containerd-shim`, the kubelet runtime, and the CNI plugin, and a cluster-wide unscoped kill on `sys_setns` terminates these runtime helpers and leaves every subsequent pod stuck in `ContainerCreating`. Scoping the kill to the `nsenter` binary, the only `setns(2)` invoker among the catalogue atomics, attributes it unambiguously to T1611.1 while leaving the container runtime untouched.

Atomic T1611.1 was repeated ten times with the enforcement policy active. A repetition is reported as blocked when both halves of the success criterion in Section 4.6 hold: the calling process is terminated before `setns(2)` returns to user space, observable as the privileged pod terminating with exit code 137 and the post-`setns` shell chain (`id; hostname; ls /etc/kubernetes; echo MARKER`) producing no output in the container log; and the kill is attributable to the matching policy, observable as a Tetragon `process_kprobe` event for `__x64_sys_setns` carrying `policy_name = sys-setns-kill` and `action = KPROBE_ACTION_SIGKILL` within the repetition’s window. Both signals must agree.

Table 5.8 reports the aggregate outcome. All ten repetitions satisfy both halves of the success criterion: every pod exits 137 (128 + SIGKILL) with no post-`setns` marker output, and every repetition has a matching `sys-setns-kill` policy event attributing the kill. End-to-end latency is tightly distributed between 2.13 and 3.03 s (mean 2.58 s). This figure is dominated by pod scheduling, image cache lookup, and `containerd-shim` setting up the container’s namespaces rather than by Tetragon’s in-kernel processing: the SIGKILL itself is delivered in the BPF program attached to

**Table 5.8:** Tetragon enforcement demonstration on T1611.1, aggregate outcome over 10 repetitions. *Exit* is the container exit code; *Marker* is 1 if the post-setns shell chain produced any output, 0 otherwise; *Policy event* records emission of a matching sys-setns-kill kprobe event with action KPROBE\_ACTION\_SIGKILL in the repetition’s window. End-to-end latency is from pod creation to the policy event landing in the export stream, dominated by pod scheduling and container startup rather than by Tetragon’s in-kernel kill path.

| Outcome (10/10 repetitions)                                 | Value                                                      |
|-------------------------------------------------------------|------------------------------------------------------------|
| Exit code                                                   | 137                                                        |
| Reason                                                      | Error                                                      |
| Marker emitted (post-setns shell chain produced any output) | 0/10                                                       |
| Matching sys-setns-kill policy event in window              | 10/10                                                      |
| End-to-end latency (s): min / mean / max                    | 2.13 / 2.58 / 3.03                                         |
| Per-rep latencies (s)                                       | 2.46, 3.03, 2.46, 3.00, 2.31, 2.60, 2.90, 2.20, 2.69, 2.13 |

the syscall, on the order of microseconds inside setns(2) before the call returns to user space.

Two construct notes qualify this demonstration. The enforcement pod uses a debian:bookworm-slim image rather than the detection pass’s alpine:3.20 (Subsection 4.2.3): Alpine’s /usr/bin/nsenter is a busybox symlink, which Tetragon’s matchBinaries filter (resolving the executable path) does not match, whereas debian ships a real nsenter; this swaps the binary behind the same shell command, not the atomic or what the policy sees in the kernel. T1611.1 was chosen because setns(2) is the entire substance of the attack, so blocking the syscall blocks it outright; generalising to composite-signal atomics would require multiple kprobes and stateful BPF logic, left to Chapter 6.

## 5.9 Cross-tool Summary

The per-tool findings consolidate into Table 5.9, which Chapter 6 draws on.

**Table 5.9:** Consolidated cross-tool summary. *DR*: aggregate detection rate over 180 repetitions (Section 5.2). *MTTD*: unweighted mean over detected repetitions (Section 5.3). *FP total* and *FP resid.*: baseline alert count over the 30 min window, before and after subtracting workload-generator artefacts (Section 5.4). *CPU* and *Mem*: per-pod baseline means (Section 5.6). *AQ*: composite alert-quality score out of 14 (Section 5.7). *Fric.*: deployment-friction count (Section 5.5). *Enforce*: per-repetition block outcome on T1611.1, “–” for no in-kernel enforcement (Section 5.8).

| Tool     | DR      | MTTD (s) | FP total | FP resid. | CPU (m) | Mem (MiB) | AQ    | Fric. | Enforce |
|----------|---------|----------|----------|-----------|---------|-----------|-------|-------|---------|
| Falco    | 177/180 | 1.10     | 31,625   | 110       | 22.8    | 93.1      | 14/14 | 0     | –       |
| Tetragon | 164/180 | 1.22     | 1,533    | 24        | 18.5    | 201.6     | 10/14 | 2     | 10/10   |
| Tracee   | 179/180 | 1.28     | 6,458    | 2,843     | 44.9    | 125.5     | 12/14 | 2     | –       |

## 6 Discussion and Conclusion

### 6.1 Synthesis and Research Questions

The results of Chapter 5 and the architectural review of Chapter 2 together address the three research questions of Section 1.3. RQ1 (architectural differences) is answered in Section 2.3. The three tools differ principally on the enforcement axis: Falco evaluates events in userspace against a domain-specific rule pack, Tracee subscribes to built-in events through Policy CRDs alongside a behavioural signature subsystem, and Tetragon attaches kprobes declaratively through TracingPolicy CRDs and uniquely supports in-kernel enforcement via `matchActions` selectors. RQ2 (detection capability, resource overhead, operational complexity on AKS) is summarised in Table 5.9: aggregate detection rates cluster tightly (Falco 98.3%, Tetragon 91.1%, Tracee 99.4%); mean MTTD differs by less than 200 ms across tools; resource overhead places all three within one order of magnitude, with memory profile the dominant differentiator (Tetragon roughly 200 MiB per pod, Falco 93 MiB); operational complexity scales with per-policy authoring effort (highest Tetragon, lowest Tracee) rather than with deployment friction. The detection rates are lower than the 100% reported by Syairozi and Arizal [14] on the same three tools, consistent with the broader catalogue used here (18 atomics across ten MITRE tactics versus three OWASP-derived scenarios) and with the inclusion of signals not covered by the tools' upstream rule and signature packs. RQ3 (practical tool-selection guidance) is answered in Section 6.2 below.

### 6.2 Tool-Selection Guidance

The cross-tool summary in Table 5.9 supports a three-way differentiation; a reader should map the operational requirement to the dimension most relevant to it.

**Falco** fits when the requirement is detection rather than enforcement, when alert metadata richness matters (severity and MITRE tagging on every alert, 14/14 on alert quality), and when an out-of-the-box rule pack is preferred over policy authoring (14/18 catalogue atomics detected by the upstream packs alone). MTTD at 1.10 s is the lowest of the three and deployment friction is nil. Falco does not block in the kernel.

**Tetragon** fits when the requirement includes preventive blocking and the team can invest in TracingPolicy authoring for detection coverage. The enforcement demonstration on T1611.1 (10/10 blocked, mean response 2.58 s dominated by pod scheduling rather than by Tetragon's in-kernel kill path) is unique among the three. Tetragon also has the lowest residual FP count after stripping workload-generator noise (24 environmental alerts) and the lowest CPU footprint (18.5 m). The tradeoffs are the highest per-policy authoring effort, no severity or MITRE tags in the alert payload, and a detection rate (91.1%) below the other two on the catalogue used here.

**Tracee** fits when the requirement emphasises coverage at low authoring cost (6/18 atomics detected at install, with the lowest per-policy authoring effort to close the gaps), when the highest aggregate detection rate is sought (99.4%), and when in-kernel enforcement is not required. The tradeoffs are the highest baseline FP residual (2,843 alerts after workload-generator stripping), the highest CPU footprint (44.9 m), and severity/MITRE tagging present only on signature findings, not on plain Policy events.

### 6.3 Limitations

Beyond the threats enumerated in Section 4.7, three limitations bear on the findings. Kubernetes API audit logs were deliberately out of scope (Subsection 4.2.2), so atomics whose primary signal lives in the audit log were excluded from the catalogue rather than counted against the tools; the comparison therefore does not address audit-log-driven capabilities such as Falco’s `k8s_audit` plugin. The resource-overhead figures (Section 5.6) come from a follow-up 15 min attack burst rather than the canonical 4 hour catalogue runs, so the attack-peak column may understate sustained-attack consumption. The enforcement evaluation (Section 5.8) is qualitative and scoped to a single high-signal atomic (T1611.1), demonstrating that Tetragon’s `matchActions` mechanism functions as designed but not establishing enforcement performance across composite-signal atomics or under load.

### 6.4 Future Work and Concluding Remarks

Three directions extend this work: integrating the AKS Diagnostic Settings audit-log stream with node-level eBPF events to close the audit-log gap; composing the tools (e.g. Tetragon for enforcement alongside Falco or Tracee for detection), excluded here by the sequential-evaluation invariant of Subsection 4.4.1; and extending enforcement across the catalogue by authoring `matchActions` selectors for composite-signal atomics, converting the qualitative demonstration of Section 5.8 into a comparative measurement.

This master’s thesis evaluated Falco, Tetragon, and Tracee on AKS against an 18-atomic catalogue covering all ten MITRE ATT&CK for Containers tactics. Across 540 attack repetitions and three 30 min baseline windows, the three tools detect at comparable rates with mean response times around one second but differ materially on policy authoring effort, alert metadata completeness, baseline FP profile, and in-kernel enforcement availability. Table 5.9 consolidates these differences and Section 6.2 maps them to the operational requirements they plausibly satisfy.

## References

- [1] Aqua Security. *Tracee: Runtime Security and Forensics using eBPF*. <https://www.aquasec.com/products/tracee/>. 2026. Accessed: June 2026.
- [2] Fábio Junior Bertinatto, Daniel Arioza Almeida, Jéferson Nobre, and Lisandro Z Granville. “EbpF-based approach to tracing system calls and predicting privilege escalation attacks.” In: *GLOBECOM 2024-2024 IEEE Global Communications Conference*. IEEE. 2024, pp. 3081–3086.
- [3] Cilium Authors. *Tetragon: eBPF-based Security Observability and Runtime Enforcement*. <https://tetragon.io/>. 2026. Accessed: June 2026.
- [4] Cloud Native Computing Foundation. *Falco: CNCF Graduated Project*. <https://www.cncf.io/projects/falco/>. 2024. Accessed: June 2026.
- [5] Gartner. *Gartner Says Worldwide IaaS Public Cloud Services Market Grew 22.5% in 2024*. <https://www.gartner.com/en/newsroom/press-releases/2025-08-06-gartner-says-worldwide-iaas-public-cloud-services-market-grew-22-point-5-percent-in-2024>. Press release. Aug. 2025. Accessed: June 2026.
- [6] Brendan Gregg. *BPF performance tools*. Addison-Wesley Professional, 2019.
- [7] Songi Gwak, Thien-Phuc Doan, and Souhwan Jung. “Container Instrumentation and Enforcement System for Runtime Security of Kubernetes Platform with eBPF.” In: *Intelligent Automation & Soft Computing* 37.2 (2023).
- [8] Jin Her, Jongseop Kim, Jinwoo Kim, and Seungsoo Lee. “An in-depth analysis of ebpf-based system security tools in cloud-native environments.” In: *IEEE Access* (2025).
- [9] Adrienn Lawson and Jeffrey Sica. *CNCF Annual Cloud Native Survey: The infrastructure of AI’s future*. Survey Report. Cloud Native Computing Foundation (CNCF). The Linux Foundation, Jan. 2026. URL: [https://www.cncf.io/wp-content/uploads/2026/01/CNCF\\_Annual\\_Survey\\_Report\\_final.pdf](https://www.cncf.io/wp-content/uploads/2026/01/CNCF_Annual_Survey_Report_final.pdf). Accessed: June 2026.
- [10] MITRE ATT&CK. *Containers Matrix*. <https://attack.mitre.org/versions/v19/matrices/enterprise/containers/>. MITRE ATT&CK Enterprise Matrix for Containers v19. 2026. Accessed: June 2026.
- [11] Red Canary. *Atomic Red Team*. <https://github.com/redcanaryco/atomic-red-team>. 2026. Accessed: June 2026.
- [12] Liz Rice. *Learning eBPF*. O’Reilly Media, Inc., 2023.
- [13] Jed Salazar and Natalia Reka Ivanko. *Security Observability with eBPF*. O’Reilly Media, Inc., 2022.

- [14] Aldien Asy Syairozi and Arizal. “Comparative Analysis of eBPF-Based Runtime Security Monitoring Tools in Monitoring and Threat Detection on Kubernetes.” In: *Proceedings of the 1st International Conference on Research and Innovations in Information and Engineering Technology - Volume 1: RITECH*. INSTICC. SciTePress, 2025, pp. 136–141. ISBN: 978-989-758-784-9. DOI: [10.5220/0014272700004928](https://doi.org/10.5220/0014272700004928).
- [15] Thomas Martijn van Vugt and Tania Malik. “A practical analysis of open-source security tools in microservice Kubernetes environments.” In: *2023 Cyber Research Conference-Ireland (Cyber-RCI)*. IEEE. 2023, pp. 1–8.

## A Evaluation Metric Definitions

The body of Section 4.6 summarises each metric in one paragraph; the construct rationale, departures from the reference paper, and unit conventions are recorded here.

### Mean Time to Detect

The mean time to detect (MTTD) is the average elapsed time between the start of an attack atomic and the first matching alert. Following Syairozi and Arizal [14],

$$\text{MTTD} = \frac{1}{n} \sum_{i=1}^n (t_{\text{alert},i} - t_{\text{start},i})$$

where  $n$  is the number of repetitions for which a matching alert was emitted,  $t_{\text{start},i}$  is the start timestamp recorded by the runner for repetition  $i$ , and  $t_{\text{alert},i}$  is the timestamp of the earliest matching alert for that repetition. Repetitions with no matching alert are excluded from the average and reported separately as misses; this departs from the reference paper, in which all repetitions detected, and accommodates the empirical misses observed in the broader catalogue used here, where not every (atomic, tool) cell detects at every repetition. MTTD is reported per atomic per tool as a mean and standard deviation, and aggregated per tool across the catalogue for the headline comparison. The unit is seconds.

### Detection Rate

The detection rate (DR) is the proportion of repetitions in which at least one matching alert was emitted. Following the standard formulation also adopted by Syairozi and Arizal [14],

$$\text{DR} = \frac{TP}{TP + FN}$$

where  $TP$  is the number of repetitions that produced at least one matching alert and  $FN$  is the number that did not. DR is reported per atomic per tool and aggregated per tool across the catalogue. The reference paper reports 100% DR for all three tools across three OWASP scenarios; the broader catalogue used here is expected to produce variation in DR across atomics, which is the principal axis of the detection comparison.

### False Positive Behaviour

The textbook false positive rate,  $FP/(FP + TN)$ , is not directly applicable to continuous time runtime monitoring because the true negative count  $TN$  is not observable; any discretisation choice

(per second, per syscall, per process) changes the value by orders of magnitude. The reference paper reports  $FPR = 0\%$  for all three tools but does not report the true negative count its denominator requires (with  $FP = 0$  the rate is 0 regardless), which makes the value uninformative for cross study comparison. This thesis instead counts the alerts emitted by each tool during a thirty minute baseline window of continuous benign traffic with no attack atomics executed (Subsection 4.2.3), and reports two quantities: the absolute count of baseline alerts per tool and the rate normalised per minute. The construct validity implications of this deviation are revisited in Section 4.7.

## Resource Overhead

CPU and memory overhead are sampled from the Kubernetes Metrics Server via `kubectl top` against the tool’s DaemonSet pods at one minute intervals throughout each session. Two summary statistics are reported per tool: the steady state average across the baseline window (the per pod resources consumed in the absence of attacks), and the peak observation during the attack phase. CPU is reported in millicores and memory in mebibytes. Because all three tools were run under identical resource limits (Subsection 4.4.3), the figures reflect what each tool actually used within an equal envelope and are not directly comparable to chart default deployments.

## Alert Quality and Contextual Enrichment

Alert quality is a qualitative dimension assessed by inspecting the structure of alerts emitted during the attack phase. For each tool, a sample of alerts (one per atomic that produced detections) is examined for the presence of seven contextual fields: rule or signature identifier, severity, process name and command line, file path or network endpoint, Kubernetes pod name and namespace, container identifier, and MITRE ATT&CK technique tag. Each field is scored as present, partial, or absent. A composite per tool score is reported alongside the breakdown, with the breakdown carrying more interpretive weight than the score because the dimensions are not equally weighted by all use cases. This metric reflects the operational utility of alerts for incident response and corresponds to the observability evaluation conducted by Van Vugt and Malik [15], extended here to a wider range of attack types.

## Operational Complexity

Operational complexity is assessed along three aspects: deployment friction, policy authoring effort, and observability defaults. Deployment friction is the count of operational obstacles encountered during the deployment of each tool (OOM kill remediation, policy name collisions, separate policy application, readiness probe workarounds), reported in Section 5.5. Policy authoring effort is the

number of distinct configuration artefacts authored for this study to bring each tool’s coverage of the catalogue to its observed level: seven custom Falco rules, nine Tetragon TracingPolicies adapted from upstream examples, and nine custom Tracee Policy CRDs. The three counts are similar in scale, which is a function of the catalogue having a finite set of detection signals rather than evidence that the three tools are equally configurable. Observability defaults capture whether each tool, in its out of the box state, emits alerts on attack patterns at all, or requires explicit configuration to do so. These three aspects feed into the tool selection guidance discussed in Section 6.2.

## **Runtime Enforcement**

Runtime enforcement is the ability to block an attack at the kernel level rather than alert on it after the fact. Of the three tools evaluated, only Tetragon supports this directly, through the `matchActions` field of `TracingPolicy` resources. Falco’s core engine emits alerts but does not block; the separate Falco Talon project provides external response orchestration and is not included in this evaluation. Tracee, in the version evaluated here, is detection only. Because the capability is not symmetric, runtime enforcement is reported as a qualitative demonstration on Tetragon rather than as a comparative metric. Atomic T1611.1 is repeated ten times with the existing `setns` `TracingPolicy` active, and success is defined as termination of the calling process before the targeted `syscall` completes, attributable to the matching policy. The asymmetry itself, that two of three tools cannot block, is treated as one of the substantive cross tool differences surfaced by the comparison.

## B MTTD per Atomic per Tool

**Table B.1:** MTTD per atomic per tool (seconds). Cell shows mean over detected repetitions; “–” if 0/10 detected.  $n$  in parentheses where  $n < 10$ .

| Atomic       | Technique | Falco      | Tetragon   | Tracee     |
|--------------|-----------|------------|------------|------------|
| T1078_003-8  | T1078.003 | 0.96       | 1.01       | 0.08       |
| T1053_002-3  | T1053.002 | 0.97       | 1.11 (n=8) | 0.07       |
| T1059_004-2  | T1059.004 | 0.95 (n=9) | 1.03 (n=9) | 0.05       |
| T1098_004-1  | T1098.004 | 0.88       | 1.01       | 0.70 (n=9) |
| T1136_001-10 | T1136.001 | 0.87       | 1.08       | 0.06       |
| T1543_002-1  | T1543.002 | 0.90       | 0.98 (n=9) | 5.89       |
| T1110_001-5  | T1110.001 | 1.00       | 1.17 (n=7) | 1.33       |
| T1552_007-3  | T1552.007 | 0.94       | 1.10       | 0.91       |
| T1611-1      | T1611     | 2.02 (n=9) | 2.25       | 1.74       |
| T1685_004-1  | T1685.004 | 2.04       | 2.18 (n=9) | 3.44       |
| T1036_004-3  | T1036.004 | 1.02       | 1.15 (n=9) | 0.10       |
| T1070_003-1  | T1070.003 | 0.86       | 1.10       | 0.85       |
| T1046-1      | T1046     | 0.95       | 1.09 (n=8) | 3.92       |
| T1069_001-1  | T1069.001 | 0.96 (n=9) | 1.06       | 1.00       |
| T1613-2      | T1613     | 1.59       | 2.24 (n=8) | 1.83       |
| T1550_001-1  | T1550.001 | 1.00       | 1.19 (n=8) | 0.06       |
| T1485-2      | T1485     | 0.83       | 0.12 (n=9) | 0.06       |
| T1496-1      | T1496     | 1.05       | 1.18       | 0.93       |

## C Per-Tool Deployment Detail

This appendix lists, per tool, the upstream rule and policy packs loaded and the custom rules, TracingPolicies, and Policy CRDs prepared for this study. The body of Subsection 4.4.2 states the headline configuration; the granular naming and per-artefact scope are recorded here.

### Falco

The modern eBPF driver, which uses CO-RE based BPF programs and requires no kernel headers or driver compilation, attached cleanly to kernel 5.15 on the AKS Ubuntu 22.04 node image. Three upstream rule packs were loaded together: the default `falco-rules` (version 5.0.0), `falco-incubating-rules` (version 6.0.0), and `falco-sandbox-rules` (version 6.0.0). Rule

auto update was disabled (`falcoctl.artifact.follow.enabled: false`) so that the rule set remained fixed for the duration of the experiment. In addition to the three upstream packs, seven custom rules were authored for this study, covering five techniques that scored zero detections against the combined upstream rule set at rule-authoring time: T1046 (network service discovery via shell driven TCP connect), T1485 (data destruction with `dd`, two rules), T1552.007 (service account token read by a file inspection tool), T1496 (resource hijacking via `yes` or `stress-ng`), and T1069.001 (local permission groups discovery via `getent` or `groups` or by reading `/etc/group`, two rules). In the canonical run the adapted T1046 atomic, which scans the cluster service network, is additionally detected by stock network rules (Contact K8S API Server From Container, System procs network activity), so upstream coverage of the final catalogue is 14/18 and four atomics (T1069.001, T1485, T1496, T1552.007) depend on the custom rules alone. Each custom rule is scoped as narrowly as the technique allows so that it contributes minimal baseline noise relative to the upstream packs.

## Tetragon

The chart's default sensors emit `process_exec` and `process_exit` events for every process lifecycle event but contain no detection policies of their own. Nine TracingPolicy resources, adapted from the upstream Tetragon example policies and scoped to the signal classes required by the catalogue, were applied on top of the default installation: `file-monitoring` (write access to `/etc/` persistence paths and read access to service account tokens), `follow-symlink` (symlink resolution onto credential and history files), `exec-sensitive-binaries` (execution of a curated set of binaries used by the catalogue atomics, scoped to the `atomic-red` namespace as a TracingPolicyNamespaced resource), `sys-setuid` (setuid syscalls), `sys-setns` (setns syscalls used by `nsenter` container escape), `sys-prctl-set-name` (`PR_SET_NAME` via `prctl`, the process masquerade primitive), `tcp-connect-egress` (TCP connect to ports 80 and 443), `security-socket-connect-unix` (`AF_UNIX` connect, used by `crictl` to reach the containerd socket), and `unlink-sensitive` (deletion of `.bash_history`, `authorized_keys`, and SSH private keys). Each policy is filtered narrowly to signals required by the catalogue so that its contribution to baseline noise is bounded. The default Tetragon memory limit of 1 GiB triggered an OOM kill during the `kprobe` attach phase once the nine policies were loaded; the limit was raised to 2 GiB and the CPU limit to 2 vCPU, matching the resource envelope applied to Falco and Tracee.

## Tracee

The runner reads events directly from the Tracee container's standard output via `kubectl logs`, the same pattern used for Falco and Tetragon. With its default policies and signature set, Tracee detected six of the eighteen catalogue atomics out of the box (T1046, T1053.002, T1110.001, T1611, T1613, T1685.004), through stock signatures such as TRC-1022 (dropped executable), TRC-101, and TRC-1015 alongside container lifecycle events from the default policy. To bring Tracee onto the same curation level as Falco and Tetragon, nine custom Policy CRDs were authored for this study and applied on top of the default installation: `audit-authorized-keys` (T1098.004, SSH `authorized_keys` access), `audit-bash-history` (T1070.003, history file access or removal), `audit-ssh-private-keys` (SSH private key access), `audit-sa-token` (T1552.007 and T1550.001, service account token reads), `exec-dd` (T1485, destructive overwrite via `dd`), `exec-cpu-burner` (T1496, resource hijacking via `yes`, `stress`, `stress-ng`, or `cpuburn`), `exec-uname-pod` (`uname` execution, the canonical first command after an `exec` into a pod), `exec-http-clients` (T1613 and T1550.001, `curl` or `wget` for API enumeration or token replay; in the canonical run it also anchors the shell-driven atomics T1036.004, T1059.004, and T1136.001), and `exec-groups-discovery` (T1069.001, local permission groups enumeration). The policy-to-atomic mapping is many-to-many rather than one per atomic: `audit-sa-token` and `exec-http-clients` each anchor several atomics, while `audit-ssh-private-keys` and `exec-uname-pod` were authored against atomics from an earlier catalogue iteration (T1552.004 and T1609, both since descoped) and are retained because their `file-open` and `process-exec` signals also support neighbouring atomics in the canonical run. All nine policies are scoped globally, with attribution to the experimental namespace performed by the runner. Each policy subscribes to Tracee's built in events, principally `security_file_open`, `sched_process_exec`, and `security_inode_unlink`, with narrow argument filters scoped to the catalogue's signals. No custom signatures were authored; the policies activate existing events rather than introduce new detection logic.